200 Machine Learning Commands

Code + theory for Python data science and machine learning

✦ **Quick start:** This guide covers 200 essential machine learning commands across pandas, scikit-learn, XGBoost, LightGBM, CatBoost, and Keras. Every command has a short explanation of what it does and when to use it, followed by a code snippet you can copy. The sections follow a real ML workflow, from loading data to tuning and saving a model.

1. Setup & Imports

#	Command	Library
1	NumPy	NumPy
2	Pandas	pandas
3	Matplotlib	Matplotlib
4	Seaborn	Seaborn
5	train_test_split	scikit-learn

◎ **Key insight:** A typical ML workflow runs in this order: import libraries, load data, explore, clean, encode and scale, split, train, evaluate, tune, save. The sections in this guide follow that flow.

1. NumPy

Library for fast numerical arrays and math operations. It is the foundation of the ML stack.

```
import numpy as np
```

2. Pandas

Handles tabular data (DataFrames). Used for loading, cleaning, and analysis.

```
import pandas as pd
```

3. Matplotlib

Basic plotting library for line, bar, scatter, and histogram charts.

```
import matplotlib.pyplot as plt
```

4. Seaborn

Built on Matplotlib. Makes statistical plots and heatmaps easy.

```
import seaborn as sns
```

5. train_test_split

Splits data into training and testing sets so you can evaluate the model on unseen data and detect overfitting.

```
from sklearn.model_selection import train_test_split
```

2. Data Loading & Exploration

#	Command	Library
6	read_csv	pandas
7	head()	pandas
8	tail()	pandas
9	shape	pandas
10	info()	pandas
11	describe()	pandas
12	dtypes	pandas
13	columns	pandas
14	nunique()	pandas
15	isnull().sum()	pandas
16	duplicated().sum()	pandas
17	value_counts()	pandas
18	corr()	pandas
19	Heatmap	Seaborn
20	Pairplot	Seaborn

◎ **Key insight:** Before any modelling, run **info()**, **isnull().sum()**, and **describe()**. These three commands reveal most data-quality problems in under a minute.

6. read_csv

Loads a CSV file into a DataFrame.

```
df = pd.read_csv("data.csv")
```

7. head()

Shows the first 5 rows for a quick look.

```
df.head()
```

8. tail()

Shows the last 5 rows. Useful for checking the end of the data.

```
df.tail()
```

9. shape

Returns (rows, columns), the size of the dataset.

```
df.shape
```

10. info()

Shows column names, data types, and non-null counts. Hints at missing values.

```
df.info()
```

11. describe()

Statistical summary: mean, std, min, max, quartiles.

```
df.describe()
```

12. dtypes

Data type of each column. Helps catch wrong types (for example, numbers stored as strings).

```
df.dtypes
```

13. columns

Lists all column names.

```
df.columns
```

14. nunique()

Number of unique values per column. Helps separate categorical from continuous features.

```
df.nunique()
```

15. isnull().sum()

Counts missing values per column. The first step of cleaning.

```
df.isnull().sum()
```

16. duplicated().sum()

Counts duplicate rows. Duplicates can bias a model.

```
df.duplicated().sum()
```

17. value_counts()

Shows the class distribution of the target and reveals class imbalance.

```
df['target'].value_counts()
```

18. corr()

Correlation between columns (-1 to +1). Helps find important features and multicollinearity.

```
df.corr(numeric_only=True)
```

19. Heatmap

Displays the correlation matrix as colors so patterns are easy to spot.

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
```

20. Pairplot

Scatter plots for every feature pair, showing relationships and distributions at once.

```
sns.pairplot(df)
```

3. Data Cleaning

#	Command	Library
21	drop_duplicates	pandas
22	dropna	pandas
23	Mean imputation	pandas
24	Median imputation	pandas
25	Mode imputation	pandas
26	Drop column	pandas

#	Command	Library
27	astype	pandas
28	rename	pandas
29	to_datetime	pandas
30	Quartiles (IQR step 1)	pandas
31	IQR outlier removal	pandas
32	SimpleImputer	scikit-learn
33	KNNImputer	scikit-learn
34	Boxplot	Seaborn
35	reset_index	pandas

> ⚠ **Warning:** Do not delete outliers or rows blindly. Check whether they are errors or real extreme values, and avoid **dropna()** when a large share of rows has missing data. Fit imputers (#32, #33) on training data only.

21. drop_duplicates

Removes repeated rows.

```
df.drop_duplicates(inplace=True)
```

22. dropna

Deletes rows with missing values. Use only when missing data is very small.

```
df.dropna(inplace=True)
```

23. Mean imputation

Fills missing values with the average. Fine for roughly normal data.

```
df['col'] = df['col'].fillna(df['col'].mean())
```

24. Median imputation

Fills with the median. Better than mean when outliers exist.

```
df['col'] = df['col'].fillna(df['col'].median())
```

25. Mode imputation

Fills with the most frequent value. Best for categorical columns.

```
df['col'] = df['col'].fillna(df['col'].mode()[0])
```

26. Drop column

Removes useless columns like IDs, which carry no predictive value.

```
df.drop(columns=['id'], inplace=True)
```

27. astype

Converts a column's data type.

```
df['col'] = df['col'].astype(int)
```

28. rename

Makes column names clean and consistent.

```
df.rename(columns={'old': 'new'}, inplace=True)
```

29. to_datetime

Converts strings to datetime so you can extract year, month, day, etc.

```
df['date'] = pd.to_datetime(df['date'])
```

30. Quartiles (IQR step 1)

Q1 and Q3 are the 25th and 75th percentiles, used to detect outliers.

```
Q1, Q3 = df['col'].quantile(0.25), df['col'].quantile(0.75)
```

31. IQR outlier removal

Keeps only values within 1.5×IQR of the quartiles. Anything outside is treated as an outlier.

```
df = df[(df['col'] >= Q1 - 1.5*(Q3-Q1)) &
        (df['col'] <= Q3 + 1.5*(Q3-Q1))]
```

32. SimpleImputer

The sklearn way of imputing (mean, median, most_frequent). Works inside pipelines.

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median')
```

33. KNNImputer

Fills missing values using the average of the nearest neighbors. Smarter than a plain mean.

```
from sklearn.impute import KNNImputer
knn = KNNImputer(n_neighbors=5)
```

34. Boxplot

Visualizes the median, quartiles, and outliers of a column.

```
sns.boxplot(x=df['col'])
```

35. reset_index

Rebuilds a clean 0..n index after rows are removed.

```
df.reset_index(drop=True, inplace=True)
```

4. Encoding & Scaling

#	Command	Library
36	get_dummies	pandas
37	LabelEncoder	scikit-learn
38	OneHotEncoder	scikit-learn
39	OrdinalEncoder	scikit-learn
40	StandardScaler	scikit-learn
41	MinMaxScaler	scikit-learn
42	RobustScaler	scikit-learn

⚠ **Warning:** Always **fit** scalers and encoders on **X_train** only, then just **transform** X_test. Fitting on the full dataset leaks test information into training (data leakage).

◎ **Key insight:** Tree-based models (Decision Tree, Random Forest, XGBoost) do not need scaling. SVM, KNN, logistic regression, and neural networks do.

36. get_dummies

One-hot encodes categorical columns into 0/1 columns. `drop_first=True` avoids the dummy variable trap.

```
df = pd.get_dummies(df, drop_first=True)
```

37. LabelEncoder

Converts categories to integers (0, 1, 2). Best for target labels, not input features.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
```

38. OneHotEncoder

The sklearn version of one-hot encoding. `handle_unknown='ignore'` prevents errors on unseen categories.

```
from sklearn.preprocessing import OneHotEncoder
ohe = OneHotEncoder(handle_unknown='ignore')
```

39. OrdinalEncoder

Encodes ordered categories (Low < Medium < High) as numbers.

```
from sklearn.preprocessing import OrdinalEncoder
oe = OrdinalEncoder()
```

40. StandardScaler

Rescales to mean 0 and std 1. Needed for SVM, KNN, and logistic regression.

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
```

41. MinMaxScaler

Scales values to the 0 to 1 range. Good for neural networks and distance-based models.

```
from sklearn.preprocessing import MinMaxScaler
mm = MinMaxScaler()
```

42. RobustScaler

Scales using the median and IQR, so it is not affected by outliers.

```
from sklearn.preprocessing import RobustScaler
rs = RobustScaler()
```

5. Feature Engineering & Selection

#	Command	Library
43	Features (X)	pandas
44	Target (y)	pandas

#	Command	Library
45	Train-test split	scikit-learn
46	Stratified split	scikit-learn
47	PolynomialFeatures	scikit-learn
48	SelectKBest	scikit-learn
49	RFE	scikit-learn
50	PCA	scikit-learn
51	Log transform	pandas
52	pd.cut	pandas
53	pd.qcut	pandas
54	apply + lambda	pandas

> ⚠ **Warning:** Feature selectors (SelectKBest, RFE) and PCA must be fitted on the training set only. Selecting features on the full data before splitting leaks information.

> ◎ **Key insight:** For classification, use **stratify=y** so train and test keep the same class proportions.

43. Features (X)

Separates the input columns from the target.

```
X = df.drop('target', axis=1)
```

44. Target (y)

The column the model has to predict.

```
y = df['target']
```

45. Train-test split

80% for training, 20% for testing. `random_state` makes the split reproducible.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

46. Stratified split

Keeps the same class ratio in both train and test sets. Important for imbalanced data.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)
```

47. PolynomialFeatures

Creates squared and interaction terms so linear models can capture curves.

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2)
```

48. SelectKBest

Keeps the top k features based on a statistical test (ANOVA F-score here).

```
from sklearn.feature_selection import SelectKBest, f_classif
selector = SelectKBest(f_classif, k=10)
```

49. RFE

Recursive Feature Elimination. Repeatedly trains a model and removes the weakest feature.

```
from sklearn.feature_selection import RFE
rfe = RFE(estimator=model, n_features_to_select=10)
```

50. PCA

Reduces dimensions while keeping maximum variance. Useful for visualization and speed.

```
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
```

51. Log transform

Reduces right skewness. `log1p` handles zeros safely.

```
df['col'] = np.log1p(df['col'])
```

52. pd.cut

Bins a numeric column into fixed ranges (for example, age groups).

```
df['bin'] = pd.cut(df['age'], bins=[0, 18, 35, 60, 100])
```

53. pd.qcut

Bins into equal-sized groups (quartiles). Each bin has roughly the same number of rows.

```
df['bin'] = pd.qcut(df['income'], q=4)
```

54. apply + lambda

Creates a new feature using a custom function on each value.

```
df['new'] = df['col'].apply(lambda x: x * 2)
```

6. Pipelines & Imbalance

#	Command	Library
55	ColumnTransformer	scikit-learn
56	Pipeline	scikit-learn
57	make_pipeline	scikit-learn
58	SMOTE	imbalanced-learn

⚠ **Warning:** Apply SMOTE to the training set only, never to the test set. Inside cross-validation, use imblearn's own Pipeline so resampling happens inside each fold.

◎ **Key insight:** A Pipeline handles the fit-on-train, transform-on-test rule automatically, which makes it the safest way to avoid data leakage.

55. ColumnTransformer

Applies different preprocessing to different columns (scale numeric, encode categorical).

```
from sklearn.compose import ColumnTransformer
ct = ColumnTransformer([('num', StandardScaler(), num_cols),
                        ('cat', OneHotEncoder(), cat_cols)])
```

56. Pipeline

Chains preprocessing and model into one object. Prevents data leakage and keeps code clean.

```
from sklearn.pipeline import Pipeline
pipe = Pipeline([('scaler', StandardScaler()),
                 ('model', LogisticRegression())])
```

57. make_pipeline

Shorter version of Pipeline that names the steps automatically.

```
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), LogisticRegression())
```

58. SMOTE

Creates synthetic minority-class samples to fix class imbalance. Apply only on training data.

```
from imblearn.over_sampling import SMOTE
X_res, y_res = SMOTE().fit_resample(X_train, y_train)
```

7. Models

#	Command	Library
59	Linear Regression	scikit-learn
60	Ridge (L2)	scikit-learn
61	Lasso (L1)	scikit-learn
62	ElasticNet	scikit-learn
63	Logistic Regression	scikit-learn
64	KNN	scikit-learn
65	Decision Tree (Classifier)	scikit-learn
66	Decision Tree (Regressor)	scikit-learn
67	Random Forest (Classifier)	scikit-learn
68	Random Forest (Regressor)	scikit-learn
69	Gradient Boosting	scikit-learn
70	SVC	scikit-learn
71	Gaussian Naive Bayes	scikit-learn
72	XGBoost	XGBoost
73	LightGBM	LightGBM
74	AdaBoost	scikit-learn
75	KMeans	scikit-learn
76	DBSCAN	scikit-learn
77	Agglomerative Clustering	scikit-learn

#	Command	Library
78	Isolation Forest	scikit-learn

> ◎ **Key insight:** Start with a simple baseline (Linear or Logistic Regression), then try Random Forest and XGBoost. If a complex model barely beats the simple one, prefer the simple one.

59. Linear Regression

Fits a straight line to predict a continuous value.

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
```

60. Ridge (L2)

Linear regression with a penalty on large coefficients. Reduces overfitting.

```
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)
```

61. Lasso (L1)

Penalty that can shrink coefficients to exactly zero, so it also does feature selection.

```
from sklearn.linear_model import Lasso
model = Lasso(alpha=0.1)
```

62. ElasticNet

Combines L1 and L2 penalties.

```
from sklearn.linear_model import ElasticNet
model = ElasticNet()
```

63. Logistic Regression

Despite the name, it is a classification model that outputs probabilities.

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=1000)
```

64. KNN

Classifies a point by the majority vote of its k nearest neighbors. Needs scaled data.

```
from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=5)
```

65. Decision Tree (Classifier)

Splits data with if-else rules. Easy to interpret, but prone to overfitting, so limit `max_depth`.

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=5)
```

66. Decision Tree (Regressor)

The same idea but predicts continuous values.

```
from sklearn.tree import DecisionTreeRegressor
model = DecisionTreeRegressor()
```

67. Random Forest (Classifier)

Many decision trees voting together (bagging). More accurate and stable than a single tree.

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100)
```

68. Random Forest (Regressor)

Averages predictions of many trees for regression.

```
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor()
```

69. Gradient Boosting

Builds trees one after another, each correcting the previous one's errors.

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier()
```

70. SVC

Finds the best boundary (hyperplane) between classes. The RBF kernel handles non-linear data.

```
from sklearn.svm import SVC
model = SVC(kernel='rbf', probability=True)
```

71. Gaussian Naive Bayes

Probabilistic classifier based on Bayes' theorem. Very fast, works well on small data.

```
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
```

72. XGBoost

Optimized gradient boosting. Very popular in competitions and on tabular data.

```
from xgboost import XGBClassifier
model = XGBClassifier()
```

73. LightGBM

Fast gradient boosting that handles large datasets efficiently.

```
from lightgbm import LGBMClassifier
model = LGBMClassifier()
```

74. AdaBoost

Boosting that gives more weight to misclassified samples in each round.

```
from sklearn.ensemble import AdaBoostClassifier
model = AdaBoostClassifier()
```

75. KMeans

Unsupervised clustering that groups data into k clusters around centroids.

```
from sklearn.cluster import KMeans
km = KMeans(n_clusters=3, random_state=42)
```

76. DBSCAN

Density-based clustering. Finds arbitrary-shaped clusters and marks noise points.

```
from sklearn.cluster import DBSCAN
db = DBSCAN(eps=0.5, min_samples=5)
```

77. Agglomerative Clustering

Hierarchical clustering that merges the closest clusters step by step.

```
from sklearn.cluster import AgglomerativeClustering
ac = AgglomerativeClustering(n_clusters=3)
```

78. Isolation Forest

Anomaly detection. Outliers are isolated quickly, so they get flagged.

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.05)
```

8. Train & Predict

#	Command	Library
79	fit()	scikit-learn
80	predict()	scikit-learn
81	predict_proba()	scikit-learn
82	score()	scikit-learn
83	coef_ and intercept_	scikit-learn
84	feature_importances_	scikit-learn

> ◎ **Key insight:** Almost every scikit-learn model follows the same pattern: **fit** on training data, **predict** on new data, then **score**. Learn it once and you can swap models freely.

79. fit()

Trains the model on the training data.

```
model.fit(X_train, y_train)
```

80. predict()

Generates predictions for new data.

```
y_pred = model.predict(X_test)
```

81. predict_proba()

Returns class probabilities. `[:, 1]` gives the probability of the positive class.

```
y_prob = model.predict_proba(X_test)[:, 1]
```

82. score()

Default metric: accuracy for classifiers, R^2 for regressors.

```
model.score(X_test, y_test)
```

83. coef_ and intercept_

Learned weights and bias of linear models. Shows each feature's effect.

```
model.coef_, model.intercept_
```

84. feature_importances_

How much each feature contributed in tree-based models.

```
model.feature_importances_
```

9. Evaluation

#	Command	Library
85	Accuracy	scikit-learn
86	Confusion Matrix	scikit-learn
87	Classification Report	scikit-learn
88	ROC-AUC	scikit-learn
89	F1 Score	scikit-learn
90	MSE	scikit-learn
91	MAE	scikit-learn
92	R^2 Score	scikit-learn
93	Cross-validation	scikit-learn
94	Silhouette Score	scikit-learn

> ⚠ **Warning:** Accuracy can look great and still be useless. If 95% of rows are one class, a model that always predicts that class scores 95%. Check recall, F1, or ROC-AUC on imbalanced data.

85. Accuracy

Percentage of correct predictions. Can be misleading on imbalanced data.

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
```

86. Confusion Matrix

Table of TP, TN, FP, FN. Shows exactly where the model makes mistakes.

```
from sklearn.metrics import confusion_matrix
confusion_matrix(y_test, y_pred)
```

87. Classification Report

Precision, recall, F1, and support for each class in one view.

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

88. ROC-AUC

Measures how well the model separates classes across all thresholds. 1.0 is perfect, 0.5 is random.

```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_test, y_prob)
```

89. F1 Score

Harmonic mean of precision and recall. Good for imbalanced data.

```
from sklearn.metrics import f1_score
f1_score(y_test, y_pred)
```

90. MSE

Average squared error for regression. Penalizes large errors heavily.

```
from sklearn.metrics import mean_squared_error
mean_squared_error(y_test, y_pred)
```

91. MAE

Average absolute error. Easy to interpret and less sensitive to outliers than MSE.

```
from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_test, y_pred)
```

92. R^2 Score

Share of variance explained by the model. 1 is perfect, 0 is no better than predicting the mean.

```
from sklearn.metrics import r2_score
r2_score(y_test, y_pred)
```

93. Cross-validation

Trains and tests on 5 different splits for a more reliable performance estimate.

```
from sklearn.model_selection import cross_val_score
cross_val_score(model, X, y, cv=5)
```

94. Silhouette Score

Measures cluster quality from -1 to 1. Higher means better-separated clusters.

```
from sklearn.metrics import silhouette_score
silhouette_score(X, km.labels_)
```

10. Tuning, Saving & Misc

#	Command	Library
95	GridSearchCV	scikit-learn
96	RandomizedSearchCV	scikit-learn
97	joblib.dump	joblib
98	joblib.load	scikit-learn
99	Permutation Importance	scikit-learn
100	pip install	pip

> ◎ **Key insight:** Use RandomizedSearchCV first to find a promising region, then refine with GridSearchCV. Tune on training data with cross-validation and touch the test set only once, at the very end.

95. GridSearchCV

Tries every combination of hyperparameters with cross-validation and picks the best.

```
from sklearn.model_selection import GridSearchCV
gs = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
gs.fit(X_train, y_train)
```

96. RandomizedSearchCV

Tries random combinations. Much faster than grid search on large search spaces.

```
from sklearn.model_selection import RandomizedSearchCV
rand = RandomizedSearchCV(model, param_dist,
                          n_iter=20, cv=5)
```

97. joblib.dump

Saves the trained model to disk so you can reuse it without retraining.

```
import joblib
joblib.dump(model, "model.pkl")
```

98. joblib.load

Loads a saved model for deployment or later predictions.

```
model = joblib.load("model.pkl")
```

99. Permutation Importance

Shuffles each feature and measures the drop in performance. Works with any model.

```
from sklearn.inspection import permutation_importance
result = permutation_importance(model, X_test, y_test)
```

100. pip install

Installs all the libraries used in this guide.

```
pip install scikit-learn xgboost lightgbm imbalanced-learn
```

11. Advanced Pandas

#	Command	Library
101	sample()	pandas
102	groupby()	pandas
103	sort_values()	pandas
104	loc / iloc	pandas
105	query()	pandas
106	concat()	pandas
107	merge()	pandas
108	pivot_table()	pandas
109	map()	pandas
110	replace()	pandas
111	String cleaning	pandas
112	select_dtypes()	pandas
113	clip()	pandas
114	category dtype	pandas
115	to_csv()	pandas

> ⚠ **Warning:** After every **merge()**, check **df.shape**. A wrong join type (**how**) or duplicate keys can silently add or drop rows.

101. sample()

Picks random rows for a quick, unbiased look at the data.

```
df.sample(5, random_state=42)
```

102. groupby()

Aggregates data per category (mean, sum, count). Core EDA tool.

```
df.groupby('category')['target'].mean()
```

103. sort_values()

Sorts rows by one or more columns.

```
df.sort_values('col', ascending=False)
```

104. loc / iloc

`loc` selects by label, `iloc` by integer position.

```
df.loc[df['age'] > 30, ['age', 'salary']]
df.iloc[0:5, 0:3]
```

105. query()

Filters rows using a readable string condition.

```
df.query("age > 30 and city == 'Delhi'")
```

106. concat()

Stacks DataFrames vertically (`axis=0`) or side by side (`axis=1`).

```
df = pd.concat([df1, df2], axis=0, ignore_index=True)
```

107. merge()

SQL-style join of two DataFrames on a common key.

```
df = pd.merge(df1, df2, on='id', how='left')
```

108. pivot_table()

Summarizes data in a matrix form, like an Excel pivot.

```
df.pivot_table(values='sales', index='region',
               columns='month', aggfunc='sum')
```

109. map()

Replaces values using a dictionary. Quick manual encoding.

```
df['gender'] = df['gender'].map({'Male': 0, 'Female': 1})
```

110. replace()

Substitutes specific values across the DataFrame.

```
df['col'] = df['col'].replace({'?': np.nan})
```

111. String cleaning

Standardizes text before encoding or modelling.

```
df['name'] = df['name'].str.lower().str.strip()
```

112. select_dtypes()

Picks numeric or categorical columns automatically.

```
num_cols = df.select_dtypes(include='number').columns
cat_cols = df.select_dtypes(include='object').columns
```

113. clip()

Caps values at limits (winsorizing) instead of deleting outliers.

```
lo, hi = df['col'].quantile([0.01, 0.99])
df['col'] = df['col'].clip(lower=lo, upper=hi)
```

114. category dtype

Saves memory and speeds up operations for low-cardinality text columns.

```
df['city'] = df['city'].astype('category')
```

115. to_csv()

Saves the cleaned data. `index=False` avoids an extra index column.

```
df.to_csv("clean_data.csv", index=False)
```

12. Visualization & Distributions

#	Command	Library
116	Histogram + KDE	Seaborn
117	countplot	Seaborn
118	scatterplot	Seaborn
119	violinplot	Seaborn
120	barplot	Seaborn
121	figure size	Matplotlib
122	skew()	pandas
123	kurt()	pandas
124	ConfusionMatrixDisplay	scikit-learn
125	RocCurveDisplay	scikit-learn

◎ **Key insight:** Plot before you model. A histogram, a countplot of the target, and a correlation heatmap usually tell you which transformation or encoding the data needs.

116. Histogram + KDE

Shows the distribution shape of one feature.

```
sns.histplot(df['col'], kde=True)
```

117. countplot

Bar chart of category counts. Great for checking class balance.

```
sns.countplot(x='target', data=df)
```

118. scatterplot

Shows the relationship between two numeric variables.

```
sns.scatterplot(x='age', y='salary', hue='target', data=df)
```

119. violinplot

Boxplot plus distribution shape per category.

```
sns.violinplot(x='target', y='age', data=df)
```

120. barplot

Shows the mean of a value per category with a confidence interval.

```
sns.barplot(x='city', y='salary', data=df)
```

121. figure size

Makes plots larger and readable.

```
plt.figure(figsize=(10, 6))
```

122. skew()

Measures asymmetry. Above +1 or below -1 means highly skewed, so consider a log transform.

```
df['col'].skew()
```

123. kurt()

Measures tail heaviness. High values suggest many outliers.

```
df['col'].kurt()
```

124. ConfusionMatrixDisplay

Plots the confusion matrix as a heatmap.

```
from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_estimator(model, X_test, y_test)
```

125. RocCurveDisplay

Plots the ROC curve directly from a fitted model.

```
from sklearn.metrics import RocCurveDisplay
RocCurveDisplay.from_estimator(model, X_test, y_test)
```

13. Advanced Preprocessing

#	Command	Library
126	Normalizer	scikit-learn
127	PowerTransformer	scikit-learn
128	QuantileTransformer	scikit-learn
129	Binarizer	scikit-learn
130	TargetEncoder	scikit-learn

#	Command	Library
131	FunctionTransformer	scikit-learn
132	VarianceThreshold	scikit-learn
133	Mutual Information	scikit-learn
134	SelectFromModel	scikit-learn
135	t-SNE	scikit-learn

> ⚠ **Warning:** TargetEncoder, PowerTransformer, and QuantileTransformer learn from the data, so fit them on the training set only. TargetEncoder in particular can leak the target if fitted on all rows. t-SNE is for plotting, not for feeding a model.

126. Normalizer

Scales each row (not column) to unit length. Common in text and cosine-similarity tasks.

```
from sklearn.preprocessing import Normalizer
nm = Normalizer()
```

127. PowerTransformer

Makes skewed data more Gaussian (Yeo-Johnson or Box-Cox).

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='yeo-johnson')
```

128. QuantileTransformer

Maps data to a uniform or normal distribution. Robust to outliers.

```
from sklearn.preprocessing import QuantileTransformer
qt = QuantileTransformer(output_distribution='normal')
```

129. Binarizer

Converts values to 0/1 using a threshold.

```
from sklearn.preprocessing import Binarizer
bz = Binarizer(threshold=0.5)
```

130. TargetEncoder

Encodes a category by the mean of the target. Useful for high-cardinality columns (needs scikit-learn 1.3+).

```
from sklearn.preprocessing import TargetEncoder
te = TargetEncoder()
```

131. FunctionTransformer

Wraps any custom function so it works inside a pipeline.

```
from sklearn.preprocessing import FunctionTransformer
ft = FunctionTransformer(np.log1p)
```

132. VarianceThreshold

Removes features with almost no variation, since they carry no information.

```
from sklearn.feature_selection import VarianceThreshold
vt = VarianceThreshold(threshold=0.01)
```

133. Mutual Information

Captures linear and non-linear dependency between a feature and the target.

```
from sklearn.feature_selection import mutual_info_classif
mi = mutual_info_classif(X, y)
```

134. SelectFromModel

Keeps features that a model (such as Lasso or Random Forest) considers important.

```
from sklearn.feature_selection import SelectFromModel
sfm = SelectFromModel(RandomForestClassifier())
```

135. t-SNE

Non-linear dimensionality reduction for visualizing clusters in 2D. For visualization only, not for modelling.

```
from sklearn.manifold import TSNE
X_2d = TSNE(n_components=2, random_state=42).fit_transform(X)
```

14. More Models & Ensembles

#	Command	Library
136	SGDClassifier	scikit-learn
137	LinearSVC	scikit-learn
138	SVR	scikit-learn
139	KNN Regressor	scikit-learn
140	Multinomial Naive Bayes	scikit-learn
141	ExtraTrees	scikit-learn

#	Command	Library
142	Gradient Boosting Regressor	scikit-learn
143	HistGradientBoosting	scikit-learn
144	XGBRegressor	XGBoost
145	LGBMRegressor	LightGBM
146	CatBoost	CatBoost
147	Bagging	scikit-learn
148	VotingClassifier	scikit-learn
149	StackingClassifier	scikit-learn
150	MLPClassifier	scikit-learn
151	Gaussian Mixture	scikit-learn
152	One-Class SVM	scikit-learn
153	Local Outlier Factor	scikit-learn
154	LDA	scikit-learn
155	CalibratedClassifierCV	scikit-learn

◎ **Key insight:** On tabular data, gradient boosting (XGBoost, LightGBM, CatBoost) is usually the strongest single model. Ensembles such as Voting and Stacking work best when the base models are different from each other.

136. SGDClassifier

Linear model trained with stochastic gradient descent. Fast on very large datasets.

```
from sklearn.linear_model import SGDClassifier
model = SGDClassifier()
```

137. LinearSVC

Faster linear version of SVM. Good for high-dimensional, sparse data like text.

```
from sklearn.svm import LinearSVC
model = LinearSVC()
```

138. SVR

Support Vector Regression. Fits within an error margin (epsilon).

```
from sklearn.svm import SVR
model = SVR(kernel='rbf')
```

139. KNN Regressor

Predicts the average of the k nearest neighbors' values.

```
from sklearn.neighbors import KNeighborsRegressor
model = KNeighborsRegressor(n_neighbors=5)
```

140. Multinomial Naive Bayes

Standard for text classification with word counts or TF-IDF.

```
from sklearn.naive_bayes import MultinomialNB
model = MultinomialNB()
```

141. ExtraTrees

Like Random Forest but with random split thresholds. Faster and often similar accuracy.

```
from sklearn.ensemble import ExtraTreesClassifier
model = ExtraTreesClassifier(n_estimators=200)
```

142. Gradient Boosting Regressor

Sequential trees for continuous targets.

```
from sklearn.ensemble import GradientBoostingRegressor
model = GradientBoostingRegressor()
```

143. HistGradientBoosting

sklearn's fast, LightGBM-style boosting. Handles missing values natively.

```
from sklearn.ensemble import HistGradientBoostingClassifier
model = HistGradientBoostingClassifier()
```

144. XGBRegressor

XGBoost for regression problems.

```
from xgboost import XGBRegressor
model = XGBRegressor(n_estimators=300, learning_rate=0.05)
```

145. LGBMRegressor

LightGBM for regression.

```
from lightgbm import LGBMRegressor
model = LGBMRegressor()
```

146. CatBoost

Boosting that handles categorical features directly, with little tuning.

```
from catboost import CatBoostClassifier
model = CatBoostClassifier(verbose=0)
```

147. Bagging

Trains many copies of a model on bootstrap samples and averages them. Reduces variance.

```
from sklearn.ensemble import BaggingClassifier
model = BaggingClassifier(n_estimators=50)
```

148. VotingClassifier

Combines different models by majority vote (hard) or averaged probability (soft).

```
from sklearn.ensemble import VotingClassifier
model = VotingClassifier(
    [('lr', lr), ('rf', rf), ('svc', svc)], voting='soft')
```

149. StackingClassifier

A meta-model learns how best to combine the predictions of base models.

```
from sklearn.ensemble import StackingClassifier
model = StackingClassifier(
    estimators=[('rf', rf), ('xgb', xgb)],
    final_estimator=LogisticRegression())
```

150. MLPClassifier

A simple neural network (multi-layer perceptron) inside sklearn.

```
from sklearn.neural_network import MLPClassifier
model = MLPClassifier(hidden_layer_sizes=(64, 32),
                      max_iter=500)
```

151. Gaussian Mixture

Soft clustering. Each point gets a probability of belonging to each cluster.

```
from sklearn.mixture import GaussianMixture
gm = GaussianMixture(n_components=3)
```

152. One-Class SVM

Learns the boundary of "normal" data to detect anomalies.

```
from sklearn.svm import OneClassSVM
oc = OneClassSVM(nu=0.05)
```

153. Local Outlier Factor

Flags points whose local density is much lower than their neighbors'.

```
from sklearn.neighbors import LocalOutlierFactor
lof = LocalOutlierFactor(n_neighbors=20)
```

154. LDA

Supervised dimensionality reduction that maximizes class separation. Also works as a classifier.

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis)
lda = LinearDiscriminantAnalysis()
```

155. CalibratedClassifierCV

Adjusts predicted probabilities so they reflect real likelihoods.

```
from sklearn.calibration import CalibratedClassifierCV
model = CalibratedClassifierCV(SVC(), cv=5)
```

15. More Evaluation & Validation

#	Command	Library
156	Precision	scikit-learn
157	Recall	scikit-learn
158	Precision-Recall Curve	scikit-learn
159	roc_curve	scikit-learn
160	Log Loss	scikit-learn

#	Command	Library
161	Balanced Accuracy	scikit-learn
162	Matthews Correlation (MCC)	scikit-learn
163	Cohen's Kappa	scikit-learn
164	RMSE	scikit-learn
165	MAPE	scikit-learn
166	cross_validate	scikit-learn
167	StratifiedKFold	scikit-learn
168	KFold	scikit-learn
169	learning_curve	scikit-learn
170	validation_curve	scikit-learn
171	cross_val_predict	scikit-learn
172	TimeSeriesSplit	scikit-learn
173	Davies-Bouldin Index	scikit-learn
174	Adjusted Rand Index	scikit-learn

◎ **Key insight:** Choose the metric by the cost of mistakes. Missing a fraud case or a disease is costly, so optimise **recall**. Raising false alarms is costly, so optimise **precision**. Need both? Use **F1**.

⚠ **Warning:** For time-ordered data never shuffle or use plain KFold. Use TimeSeriesSplit (#172), or the model will learn from the future.

156. Precision

Of all predicted positives, how many were correct. Important when false alarms are costly.

```
from sklearn.metrics import precision_score
precision_score(y_test, y_pred)
```

157. Recall

Of all actual positives, how many were found. Important when misses are costly (fraud, disease).

```
from sklearn.metrics import recall_score
recall_score(y_test, y_pred)
```

158. Precision-Recall Curve

Shows the trade-off across thresholds. Better than ROC for imbalanced data.

```
from sklearn.metrics import precision_recall_curve
p, r, t = precision_recall_curve(y_test, y_prob)
```

159. roc_curve

Gives the false positive and true positive rates for plotting a custom ROC.

```
from sklearn.metrics import roc_curve
fpr, tpr, thr = roc_curve(y_test, y_prob)
```

160. Log Loss

Penalizes confident wrong probabilities. Lower is better.

```
from sklearn.metrics import log_loss
log_loss(y_test, y_prob)
```

161. Balanced Accuracy

Average recall per class. Fair on imbalanced data.

```
from sklearn.metrics import balanced_accuracy_score
balanced_accuracy_score(y_test, y_pred)
```

162. Matthews Correlation (MCC)

A single balanced score from -1 to 1. Reliable even with skewed classes.

```
from sklearn.metrics import matthews_corrcoef
matthews_corrcoef(y_test, y_pred)
```

163. Cohen's Kappa

Agreement between predictions and truth, corrected for chance.

```
from sklearn.metrics import cohen_kappa_score
cohen_kappa_score(y_test, y_pred)
```

164. RMSE

Square root of MSE, in the same units as the target.

```
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

165. MAPE

Average percentage error. Easy for business stakeholders to understand.

```
from sklearn.metrics import mean_absolute_percentage_error
mean_absolute_percentage_error(y_test, y_pred)
```

166. cross_validate

Cross-validation with multiple metrics and train scores in one call.

```
from sklearn.model_selection import cross_validate
cross_validate(model, X, y, cv=5,
               scoring=['accuracy', 'f1'],
               return_train_score=True)
```

167. StratifiedKFold

K-fold that preserves class ratios in every fold.

```
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

168. KFold

Plain K-fold splitting for regression or balanced data.

```
from sklearn.model_selection import KFold
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

169. learning_curve

Shows performance vs training size. Diagnoses overfitting and underfitting.

```
from sklearn.model_selection import learning_curve
sizes, train_sc, val_sc = learning_curve(model, X, y, cv=5)
```

170. validation_curve

Shows performance as one hyperparameter changes.

```
from sklearn.model_selection import validation_curve
tr, va = validation_curve(
    model, X, y, param_name='max_depth',
    param_range=range(1, 15), cv=5)
```

171. cross_val_predict

Gets out-of-fold predictions for every row. Useful for error analysis and stacking.

```
from sklearn.model_selection import cross_val_predict
y_cv = cross_val_predict(model, X, y, cv=5)
```

172. TimeSeriesSplit

Cross-validation that respects time order, so training data always precedes test data.

```
from sklearn.model_selection import TimeSeriesSplit
tscv = TimeSeriesSplit(n_splits=5)
```

173. Davies-Bouldin Index

Clustering quality metric. Lower is better.

```
from sklearn.metrics import davies_bouldin_score
davies_bouldin_score(X, km.labels_)
```

174. Adjusted Rand Index

Compares cluster labels with true labels. 1 is a perfect match.

```
from sklearn.metrics import adjusted_rand_score
adjusted_rand_score(y_true, km.labels_)
```

16. Tuning, Imbalance & Interpretation

#	Command	Library
175	DummyClassifier	scikit-learn
176	class_weight	scikit-learn
177	RandomUnderSampler	imbalanced-learn
178	Optuna	Optuna
179	Early stopping (XGBoost)	scikit-learn
180	SHAP	SHAP
181	pickle	pickle
182	get_params / set_params	scikit-learn
183	best_params_ / best_score_	scikit-learn
184	Pipeline diagram	scikit-learn
185	n_jobs=-1	scikit-learn

> ⚠ **Warning:** Early stopping needs its own validation set (X_val, y_val) carved out of the training data. If you use the final test set for it, your test score becomes optimistic.

> ◎ **Key insight:** Always build a baseline first (#175). If your model cannot beat a DummyClassifier, the problem is the data or the features, not the algorithm.

175. DummyClassifier

A baseline that ignores features. Your real model must beat it.

```
from sklearn.dummy import DummyClassifier
base = DummyClassifier(strategy='most_frequent')
base.fit(X_train, y_train)
```

176. class_weight

Gives more weight to the minority class without resampling.

```
model = RandomForestClassifier(class_weight='balanced')
```

177. RandomUnderSampler

Removes majority-class rows to balance the data. Apply on training data only.

```
from imblearn.under_sampling import RandomUnderSampler
rus = RandomUnderSampler()
X_res, y_res = rus.fit_resample(X_train, y_train)
```

178. Optuna

Smart hyperparameter search that learns from previous trials.

```
import optuna
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=50)
```

179. Early stopping (XGBoost)

Stops boosting when validation score stops improving, which prevents overfitting.

```
model = XGBClassifier(n_estimators=1000,
                      early_stopping_rounds=20)
model.fit(X_train, y_train, eval_set=[(X_val, y_val)])
```

180. SHAP

Explains how much each feature pushed each individual prediction.

```
import shap
explainer = shap.TreeExplainer(model)
shap.summary_plot(explainer.shap_values(X_test), X_test)
```

181. pickle

Python's built-in way to save models (joblib is usually better for large arrays).

```
import pickle
pickle.dump(model, open("model.pkl", "wb"))
```

182. get_params / set_params

Inspect or change hyperparameters programmatically.

```
model.get_params()
model.set_params(max_depth=7)
```

183. best_params_ / best_score_

Best result after GridSearchCV or RandomizedSearchCV.

```
gs.best_params_, gs.best_score_, gs.best_estimator_
```

184. Pipeline diagram

Displays pipelines as a visual diagram in notebooks.

```
from sklearn import set_config
set_config(display='diagram')
```

185. n_jobs=-1

Uses all CPU cores for faster training and searching.

```
model = RandomForestClassifier(n_jobs=-1)
```

17. Deep Learning (Keras)

#	Command	Library
186	Sequential model	Keras / TensorFlow
187	Dense layer	Keras / TensorFlow
188	compile()	scikit-learn

#	Command	Library
189	fit() with epochs	Keras / TensorFlow
190	evaluate()	scikit-learn
191	Dropout	Keras / TensorFlow
192	EarlyStopping	Keras / TensorFlow
193	model.save()	scikit-learn
194	Conv2D	Keras / TensorFlow
195	LSTM	Keras / TensorFlow

⚠ **Warning:** Neural networks need scaled inputs (StandardScaler or MinMaxScaler). Unscaled features make training slow and unstable.

◎ **Key insight:** For plain tabular data, gradient boosting usually matches or beats neural networks with far less effort. Use deep learning mainly for images, text, audio, and sequences.

186. Sequential model

A simple stack of layers.

```
from tensorflow.keras.models import Sequential
model = Sequential()
```

187. Dense layer

Fully connected layer. The building block of neural networks.

```
from tensorflow.keras.layers import Dense
model.add(Dense(64, activation='relu',
                input_shape=(X_train.shape[1],)))
```

188. compile()

Sets the optimizer, loss function, and metrics.

```
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

189. fit() with epochs

Trains the network. One epoch is one full pass over the data.

```
history = model.fit(X_train, y_train, epochs=50,
                    batch_size=32, validation_split=0.2)
```

190. evaluate()

Returns loss and metrics on the test set.

```
model.evaluate(X_test, y_test)
```

191. Dropout

Randomly turns off neurons during training to reduce overfitting.

```
from tensorflow.keras.layers import Dropout
model.add(Dropout(0.3))
```

192. EarlyStopping

Stops training when validation loss stops improving.

```
from tensorflow.keras.callbacks import EarlyStopping
es = EarlyStopping(monitor='val_loss', patience=5,
                   restore_best_weights=True)
```

193. model.save()

Saves architecture and weights together.

```
model.save("model.keras")
```

194. Conv2D

Convolution layer that detects patterns in images such as edges and shapes.

```
from tensorflow.keras.layers import Conv2D
model.add(Conv2D(32, (3, 3), activation='relu'))
```

195. LSTM

Recurrent layer that remembers long-term patterns in sequences like text and time series.

```
from tensorflow.keras.layers import LSTM
model.add(LSTM(50, return_sequences=False))
```

18. NLP & Time Series

#	Command	Library
196	TfidfVectorizer	scikit-learn
197	CountVectorizer	scikit-learn
198	Datetime features	pandas
199	Rolling mean	pandas
200	shift() (lag features)	pandas

> ⚠ **Warning:** For time series, never use a random **train_test_split**. Split by time (or use TimeSeriesSplit, #172) so future data never leaks into training. Fit text vectorizers on the training text only.

196. TfidfVectorizer

Converts text to numbers by weighting words that are frequent in a document but rare overall.

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(stop_words='english',
                        max_features=5000)
X_text = tfidf.fit_transform(df['text'])
```

197. CountVectorizer

Bag-of-words. Counts how often each word appears.

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer(ngram_range=(1, 2))
```

198. Datetime features

Extracts year, month, weekday, etc. for time-based patterns.

```
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['weekday'] = df['date'].dt.dayofweek
```

199. Rolling mean

Smooths a series and captures the recent trend.

```
df['rolling_7'] = df['sales'].rolling(window=7).mean()
```

200. shift() (lag features)

Uses past values as features for forecasting.

```
df['lag_1'] = df['sales'].shift(1)
```

At a Glance

> ⚑ **At a glance:** Golden rules: split first, fit preprocessing on training data only, start with a baseline, pick a metric that fits the problem, tune with cross-validation, and touch the test set once.

#	Section	Commands	What you learn
1	Setup & Imports	1 to 5	Core libraries every ML project starts with
2	Data Loading & Exploration	6 to 20	Understand shape, types, missing values, and correlations first
3	Data Cleaning	21 to 35	Remove duplicates, fix types, handle missing values and outliers
4	Encoding & Scaling	36 to 42	Convert categories to numbers and put features on a common scale
5	Feature Engineering & Selection	43 to 54	Split data correctly and create or select the most useful features
6	Pipelines & Imbalance	55 to 58	Pipelines prevent leakage; SMOTE balances classes (train only)
7	Models	59 to 78	Regression, classification, boosting, clustering, and anomaly detection
8	Train & Predict	79 to 84	The fit, predict, score pattern is the same for nearly every model
9	Evaluation	85 to 94	Pick the metric that matches the problem, not just accuracy
10	Tuning, Saving & Misc	95 to 100	Tune with cross-validation, then save the best model
11	Advanced Pandas	101 to 115	Group, join, pivot, filter, and reshape data like a pro
12	Visualization & Distributions	116 to 125	Use plots to spot skew, imbalance, outliers, and model errors
13	Advanced Preprocessing	126 to 135	Fix skew, encode high-cardinality columns, and prune weak features
14	More Models & Ensembles	136 to 155	Specialised models, boosting variants, ensembles, and anomaly detectors
15	More Evaluation & Validation	156 to 174	Precision, recall, curves, error metrics, and the right CV strategy
16	Tuning, Imbalance & Interpretation	175 to 185	Baselines, imbalance handling, smarter tuning, and model explanation
17	Deep Learning (Keras)	186 to 195	Build, train, regularise, and save neural networks
18	NLP & Time Series	196 to 200	Turn text and dates into features without leaking the future